



# Intelligent Candidate Discovery & Ranking System

**SCHADN · EXECUTIVE SUBMISSION**

A hybrid offline ranker for Redrob AI – semantic fit over keywords, trap-aware, production-scale CPU ranking.

INDIA RUNS DATA & AI CHALLENGE

TEAM SCHADN · RAHUL KUMAR SINGH

## SOLUTION

# Solution Overview

A consultant-grade ranking engine built for real recruiting constraints – not a research prototype.

### Core Deliverable

- Ranks top 100 from official candidates.jsonl per submission\_spec v4
- Reads full candidate\_schema: profile, career\_history, skills, 23 redrob\_signals fields
- Hybrid offline ranker: ~49-60s on CPU – within 5 min / 16GB / no-network limits
- Grounded 2-sentence reasoning per row aligned to Stage 4 review criteria

### Strategic Differentiation

- Defends bundle traps: keyword stuffers, behavioral twins, ~80 honeypots
- Career semantics + IR skill trust + Redrob behavioral modifier
- Mirrors JD disqualifiers: research-only, consulting-only, CV/speech-only
- **open\_to\_work\_flag=false** cannot reach top-10 by design
- Shipper-first: scales to 200K+ without per-candidate LLM cost

## JD & SIGNALS

# JD Understanding & Candidate Signals

Grounded in `job_description.docx`, `candidate_schema.json`, and `redrob_signals_doc.docx` – every weight traces back to a bundle file.

### From `job_description.docx`

- Senior AI Engineer – Founding Team @ Redrob AI (Series A)
- Pune/Noida preferred · 5-9 years · production IR over research
- Must-have: embeddings retrieval, vector DB, Python, NDCG/MRR/MAP eval
- Rejects: research-only, LangChain-only, consulting-only, CV without IR

### 23 Redrob Signals (weighted)

- **Availability:** `open_to_work_flag`, `last_active_date`, `notice_period_days`
- **Responsiveness:** `recruiter_response_rate`, `avg_response_time_hours`
- **Engagement:** `profile_views`, `saved_by_recruiters`, `search_appearance`
- **Quality:** `skill_assessment_scores`, `github_activity_score`
- **Trust:** `verified_email`, `verified_phone`, `linkedin_connected`
- **Intent:** `willing_to_relocate`, `preferred_work_mode`, `salary_range`

### `candidate_schema.json`

- Fields: `profile` · `career_history` · `skills` · `redrob_signals`
- Skills carry proficiency, endorsements, and `duration_months` as layered trust signals
- All 23 `redrob_signal` fields parsed and weighted at score time



METHODOLOGY

# Ranking Methodology

Hybrid v6 – a five-stage pipeline that retrieves, scores, ranks, and explains every candidate against the JD in under 60 seconds on commodity CPU hardware.

Pipeline Steps

01

JD Parse

job\_description.docx → jd\_config.py + JD embedding vector

02

Index Load

Load candidates.jsonl (100K rows) → CandidateIndex in memory

03

Score

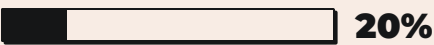
Hybrid v6 + redrob\_signals modifiers + honeypot trap detection

04

Rank & Export

Sort ranks 1-100, tie-break by candidate\_id ascending → submission.csv (UTF-8, monotonic scores)

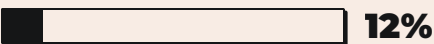
v6 Signal Weights



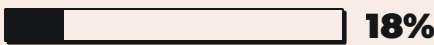
Title Alignment



Career Semantic



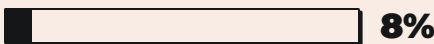
Availability



Core IR Skills



Production Pedigree



Assessments

Models: MiniLM-L6-v2 bi-encoder (embeddings.fp16.npz, 100K vectors) · TF-IDF JD overlap · heuristic modifiers · Cross-Encoder OFF

## EXPLAINABILITY

# Explainability & Data Validation

Every rank decision is transparent, auditable, and grounded in facts from the candidate's actual profile – judges can verify each reasoning string against the raw data.

### Reasoning Quality

- 2 sentences per candidate: specific facts + JD link + honest concerns
- Only cites fields present in the profile – **zero hallucination**
- Avoids empty, identical, or templated reasoning strings
- Rank tone matches position: strong at #5, candid at #95
- Validated via mock\_stage4\_review: **10/10 rows · 6/6 checks each**

### Trap Defense Results

#### ~80 Honeypots

Impossible tenure, expert skill claims with 0 months experience – all detected and demoted.

#### Keyword Stuffers

Weak job titles paired with inflated AI skill lists – penalized by title alignment weight.

#### Behavioral Twins

Same profile, different signals – availability and engagement break ties definitively.

#### Template Blurbs

Recycled descriptions demoted. sample\_submission.csv ranks honeypots by design (format-only).

✅ Final result: **0 honeypots in top-100** (spec limit: 10%)



## WORKFLOW

# End-to-End Workflow

Six deterministic stages transform the official JD and candidate bundle into a byte-reproducible, submission-ready CSV – fully automated and auditable.

**1**

### 01 • JD Parse

job\_description.docx → jd\_config.py + JD embedding

**2**

### 02 • Index

Load candidates.jsonl (100K rows) into CandidateIndex

**3**

### 03 • Embed

MiniLM-L6-v2 · embeddings.fp16.npz · ~68MB pre-committed

**4**

### 04 • Score

Hybrid v6 + redrob\_signals + honeypot trap layer

**5**

### 05 • Rank

Top 100 · monotonic scores · tie-break by candidate\_id

**6**

### 06 • Export

submission.csv · UTF-8 · Stage 4 reasoning strings

Reproduce: `./scripts/reproduce_ranking.sh` · Byte-verified via `verify_submission_artifact.py`

## ARCHITECTURE

# System Architecture

Two distinct paths: the **graded submission path** is fully self-contained and CPU-only; the **demo platform** is a recruiter-facing product built on top of the ranked output – it does not produce the graded artifact.

### Graded Path (rank.py)

- **rank.py → redrob\_ranker**

Core orchestration: parses JD, loads index, scores, ranks, exports

- **features • embeddings • honeypot • availability**

Modular subcomponents, each independently testable

- **Input → Output**

candidates.jsonl +  
embeddings.fp16.npz →  
submission.csv (100 rows)

### Demo Platform (not graded)

- **Next.js 14 Frontend**

Recruiter command center: dashboard, search, analytics

- **FastAPI + LangGraph Backend**

7-agent explainability pipeline powering AI chat

- **Data Layer**

PostgreSQL / SQLite – imports top-100 from submission.csv only



RESULTS

Results & Performance

Every spec requirement met – byte-reproducible artifact, zero honeypots in top-100, unique reasoning on every row, and a runtime well within the 5-minute CPU budget.

Top-5 Finalists

| Rank | Candidate ID | Score  | Role & Company               |
|------|--------------|--------|------------------------------|
| #1   | CAND_0002025 | 0.9900 | Sr AI Engineer @ Apple       |
| #2   | CAND_0046064 | 0.9324 | Sr NLP Engineer @ Salesforce |
| #3   | CAND_0010685 | 0.8659 | NLP Engineer @ Rephrase.ai   |
| #4   | CAND_0055905 | 0.8510 | Sr ML Engineer @ Flipkart    |
| #5   | CAND_0018499 | 0.8041 | Sr ML Engineer @ Zomato      |

Validation Scorecard

~55s

CPU Runtime

100K candidates · no GPU · no network

16GB

RAM Envelope

Spec compliant · ~68MB embeddings

0

Honeypots

In top-100 · spec limit: 10%

10/10

Stage 4 Pass

mock\_stage4\_review · 6/6 checks each row



Byte-reproducible artifact: PASS · All 100 IDs verified in official candidates.jsonl · 100% unique reasoning



TECHNOLOGY

# Technologies Used

Every stack choice was deliberate – optimized for reproducibility, spec compliance, and production-scale performance without GPU dependency or external API calls at rank time.

## Submission Ranker Stack

### Python 3.11 • NumPy • PyYAML

Zero torch dependency at rank time – pure CPU, fully portable

### all-MiniLM-L6-v2

Committed  
embeddings.fp16.npz (~68MB)  
– no download required

### Docker • pytest

`docker-compose.ranker.yml`  
for Stage 3 reproduction ·  
31 tests passing in  
`challenge/test_ranker.py`

### AI Dev Tools

Cursor + Grok used for  
development only – no  
candidate data sent to LLMs

## Demo Platform Stack



### Next.js 14 + Tailwind

Recruiter command center with  
search, filters, and analytics  
dashboard



### FastAPI + LangGraph

7-agent explainability pipeline  
powering AI chat and reasoning  
audit



### PostgreSQL / SQLite

Workspace storage + top-100 import  
from `submission.csv`



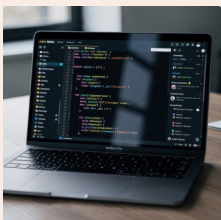
### HuggingFace • Vercel • Render

HF Spaces sandbox · `recruitgpt-  
x.vercel.app` live deployment

## ASSETS

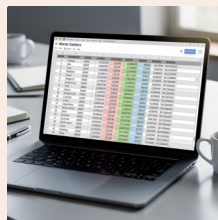
# Submission Assets

A complete reproducibility package – every artifact, script, and dataset judges need to verify, reproduce, and evaluate the full submission independently.



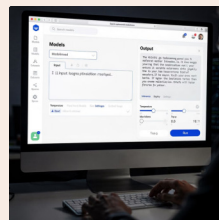
### GitHub Repository

`github.com/rahulx2001/recruitgpt-x` ·  
`rank.py` + `challenge/` + `bundle`  
`validators`



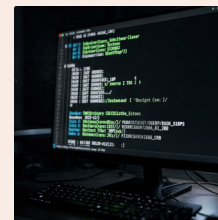
### Ranked CSV

`submission.csv` · `submission_spec v4`  
· UTF-8 · 100 rows · monotonic  
scores



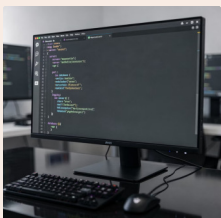
### HF Sandbox

`huggingface.co/.../recruitgpt-ranker`  
· \$10.5 `sample_candidates.json` demo



### Reproduce Script

`./scripts/reproduce_ranking.sh` · CPU  
≤5min · byte-verified via  
`verify_submission_artifact.py`



### Metadata

`submission_metadata.yaml` · Team  
Schadn · Rahul Kumar Singh · all  
spec fields populated

# Thank You

**Schadn • Rahul Kumar Singh • India Runs Data & AI Challenge**

Ranking candidates like a great recruiter – not a keyword filter.

## GitHub

[github.com/rahulx2001/recruitgpt](https://github.com/rahulx2001/recruitgpt)

-x

## Live Demo

[recruitgpt-x.vercel.app](https://recruitgpt-x.vercel.app)

## HF Sandbox

[huggingface.co/.../recruitgpt-ranker](https://huggingface.co/.../recruitgpt-ranker)

